Cross-Domain 2D Object Detection: A Post-Processing Failure Audit

Harshit Jangam

CMPE 249, Homework: 2D Object Detection — Track A, Experimental Model Evaluation

Abstract

This report evaluates four COCO-pretrained 2D object detector configurations, spanning three detector families, without any fine-tuning, on a shared three-class protocol of person, car, and truck. The source domain is COCO val2017 and the target domain is KITTI. Rather than reporting a single aggregate score, the study combines cross-domain metrics, scale and per-class breakdowns, latency and memory profiling, TIDE error decomposition, a density-stratified recall analysis, and a fixed-weight NMS-IoU intervention across eighteen saved runs. All reported metrics derive from deterministic 1,000-image saved prediction sets. The central finding is that relaxing final-stage non-maximum suppression recovers at most 0.77 mAP points against a cross-domain drop of roughly 14.3 points, bounding the post-processing contribution to approximately five percent of the domain gap. A stratified recall analysis confirms that the recovered accuracy is concentrated in same-class overlapping objects, which identifies the mechanism of the effect while also showing its limited scale. The result is a controlled failure audit, not a claim of a new detector or of state-of-the-art performance.

1. Question and Hypothesis

When a COCO-pretrained detector is transferred to an autonomous-driving domain without retraining, how much of the resulting performance loss can be attributed to final duplicate suppression rather than to upstream localization, background, classification, or recall failures? Post-processing is an attractive explanation because it is the one stage a practitioner can change without retraining, and because non-maximum suppression thresholds are tuned on the source domain and then carried unchanged into the target domain. Whether that inherited threshold is actually responsible for a material share of the degradation is an empirical question that aggregate mAP alone cannot answer.

The hypothesis under test is stated as a quantitative bound rather than a directional claim. Final-stage NMS operates after the detector has already produced its candidate set, so it can only remove or retain existing boxes. It can therefore recover accuracy where a correct detection was suppressed as a duplicate, but it cannot create a missing detection, improve a poorly localized box, or remove a hallucinated one. The prediction is that sweeping this single threshold across its usable range will recover a small, bounded fraction of the cross-domain gap, and that the residual will remain dominated by upstream error types. The intervention is designed to measure the size of that bound, not merely to confirm its sign.

2. Experimental Protocol

2.1 Models and detector families

Four configurations span three detector families. YOLO11s and YOLO11m are one-stage dense-prediction models with an anchor-free decoupled head and class-aware NMS at inference. Faster R-CNN R50-FPN v2 is a two-stage region-based detector in which a region proposal network supplies candidates to an RoI head, with suppression applied at both stages. RT-DETR R50 is a query-based set-prediction model that requires no final NMS at all, because one-to-one Hungarian matching during training removes duplicates structurally. Including two YOLO sizes provides a within-family capacity control, while RT-DETR provides the NMS-free reference point that makes the post-processing intervention interpretable.

Configuration	Family	Input preprocessing	Baseline post-processing
YOLO11s	One-stage dense	Letterbox 640 × 640	Class-aware NMS, IoU 0.70
YOLO11m	One-stage dense	Letterbox 640 × 640	Class-aware NMS, IoU 0.70
Faster R-CNN R50-FPN v2	Two-stage region	Min side 800, max side 1333	RoI NMS IoU 0.50; RPN fixed
RT-DETR R50	Query-based	Processor resize 640 × 640	Set prediction; no final NMS

Table 1. Model configurations, input preprocessing, and baseline post-processing settings.

2.2 Datasets, taxonomy, and splits

COCO val2017 is the source domain and KITTI is the target domain. Both are filtered to the shared three-class taxonomy of person, car, and truck. Classes present in only one dataset are excluded rather than heuristically remapped. This choice follows directly from a documented failure in the course starter harness, where folding COCO person and bicycle into a combined KITTI-style cyclist class produces an average precision near zero because the box

geometry of the merged class does not match either source annotation [10]. Admitting a synthetic class whose measured AP is an artifact of the mapping rather than of detector behavior would contaminate a failure audit, so the protocol accepts a narrower label space in exchange for a clean one.

The three retained classes are not equally clean. KITTI reserves the Truck label for large commercial vehicles and assigns pickups and vans to Car and Van, whereas the COCO truck category includes pickups. The truck class therefore carries a known definitional boundary in addition to a small instance count, and conclusions drawn from it are qualified accordingly throughout. The alternative of merging car and truck into a single vehicle class, as the starter harness does in its default preset, would avoid this boundary but would also discard the per-class resolution that the failure analysis depends on.

KITTI uses a generated 1,500-image held-out split. The officially labeled training identifiers are hash-ranked with seed 2026, the selected identifiers are sorted, and the first 1,000 are evaluated; the remaining 500 are unused reserves retained for a future extension. The split is reproducible from `scripts/cloud/prepare_kitti_split.py` and is a project split, not a published KITTI benchmark split. Because no model is trained or fine-tuned, train-to-test leakage does not apply. Hash ranking is used to avoid a raw contiguous frame prefix, which would otherwise concentrate the sample in a small number of recording sessions; the report makes no independent-frame statistical claim beyond that.

2.3 Evaluation settings and what is held fixed

Each run evaluates a deterministic prefix of 1,000 images, applies a score floor of 0.001, saves at most 300 candidates per image, and evaluates at most 100 detections per image. The low score floor is deliberate: mean average precision ranks detections rather than thresholding them, so a conventional operating threshold such as 0.3 truncates the low-score tail of the precision-recall curve and depresses AP for reasons unrelated to detector quality. Object-scale labels are computed in original annotation coordinates rather than post-resize coordinates, so the small, medium, and large bands describe the physical objects rather than each model's internal input geometry.

The KITTI reference image size is 1242 × 375. The Faster R-CNN transform scales each image subject to the 800-pixel short-side and 1333-pixel long-side bounds, while the two YOLO configurations and RT-DETR receive 640 × 640 tensors. This is intentionally a checkpoint-native comparison rather than a resolution-matched one, since forcing a two-stage detector into a square letterbox would introduce a confound of its own. The consequence is stated explicitly in Section 4 and revisited in Section 7. Evaluation annotations, taxonomy, score floor, candidate cap, metric cap, image manifest, batch size, and baseline GPU type are held fixed across all configurations.

Baseline measurements were executed on a single RTX A6000 48 GB cloud GPU. The NMS threshold sweep was executed on an RTX 4090 24 GB after a pod recovery. Only baseline latency and memory values are therefore used for cross-model efficiency comparison, and sweep latency is never compared across the two devices. Accuracy metrics are unaffected by this split. The baseline NMS settings are independently validated by the sweep itself: each baseline mAP falls between the values at its two adjacent sweep thresholds, which confirms that the harness is applying the documented default rather than a silently overridden one.

2.4 Commands and configuration

Every run in this study is driven from a single evaluation entry point with an explicit configuration, so that a run can be reproduced from the command alone. Table 2 lists the resolved configuration shared by all eighteen runs, and Listing 1 gives the command forms for the KITTI split generation, the baseline matrix, the ten NMS sweep points, and the report build, together with one expanded evaluator invocation exactly as recorded in a run manifest. Each run writes a manifest holding its resolved arguments, the evaluated image identifier list, the library versions, and a SHA-256 hash of its saved prediction records, so the command and the artifact can be checked against one another.

Parameter	Resolved value
Source split	COCO val2017, first 1,000 image IDs in ascending order
Target split	KITTI, seed-2026 hash-ranked 1,500-image holdout, first 1,000 evaluated
Taxonomy	person, car, truck; non-shared labels excluded, never remapped
Detection caps	Score floor 0.001; 300 candidates saved per image; 100 evaluated per image
YOLO11s / YOLO11m	yolo11s.pt / yolo11m.pt, letterbox 640 × 640, final NMS IoU 0.70
Faster R-CNN R50-FPN v2	COCO_V1 weights, min side 800 / max side 1333, RoI NMS IoU 0.50
RT-DETR R50	PekingU/rtdetr_r50vd, processor resize 640 × 640, no final NMS
Sweep and run count	NMS 0.30 / 0.45 / 0.60 / 0.75 / 0.90, final stage only; 18 runs (8 baseline + 10 sweep)

Table 2. Resolved run configuration shared across all eighteen evaluation runs.

```
# Whole production matrix: 20-image pilot, 8 baselines, 10 NMS runs
bash scripts/cloud/run_production_matrix.sh
```

```
# 1. Reproducible KITTI holdout split (hash-ranked, seed 2026)
python scripts/cloud/prepare_kitti_split.py \
    --root /workspace/detaudit-data/kitti --val-count 1500 --seed 2026

# 2. Baseline matrix wrapper, run once per model and dataset
MAX_IMAGES=1000 OUTPUT_ROOT=/workspace/detaudit-outputs \
bash scripts/cloud/run_evaluation.sh yolo11s-coco-1000 yolo:yolo11s.pt coco

# ... and likewise for yolo11m, torchvision:fasterrcnn_resnet50_fpn_v2,
#     hf_detr:PekingU/rtdetr_r50vd, each on coco and kitti

# Expanded evaluator command recorded in each run manifest
python -m detaudit.run \
    --upstream-root /workspace/DeepDataMiningLearning \
    --model yolo:yolo11s.pt --dataset coco \
    --root /workspace/detaudit-data/coco \
    --out-dir /workspace/detaudit-outputs/yolo11s-coco-1000 \
    --split val --score-floor 0.001 --max-detections 300 \
    --max-images 1000 --yolo-imgsz 640 --device cuda

# 3. Ten NMS sweep points; only --nms-iou is added
for threshold in 0.3 0.45 0.6 0.75 0.9; do
  suffix=${threshold/./}
  MAX_IMAGES=1000 NMS_IOU="${threshold}" OUTPUT_ROOT=/workspace/detaudit-outputs \
  bash scripts/cloud/run_evaluation.sh \
      "yolo11s-kitti-nms-${suffix}" yolo:yolo11s.pt kitti
  MAX_IMAGES=1000 NMS_IOU="${threshold}" OUTPUT_ROOT=/workspace/detaudit-outputs \
  bash scripts/cloud/run_evaluation.sh \
      "fasterrcnn-kitti-nms-${suffix}" \
      torchvision:fasterrcnn_resnet50_fpn_v2 kitti
done

# 4. Metrics, TIDE, strata, figures and report from saved records only
.report-venv/bin/python scripts/analysis/build_submission.py
```

Listing 1. Commands for split generation, the baseline matrix, the NMS sweep, and the report build, as recorded in the run manifests and the repository command log.

Three details of the interface are worth recording because they are not visible in the flag names. The taxonomy is not a command-line option: the runner instantiates the fixed three-class `Driving3Taxonomy`, named `coco_kitti3`, so the label space cannot drift between runs. The flag `--max-detections 300` is the saved-candidate cap, while COCO metric evaluation uses a fixed 100 detections per image, recorded as `coco_max_detections` in each manifest. There is no flag for saving records: every run writes `records.json`, `metrics.json`, `manifest.json`, `strata.json` and the TIDE report automatically, which is why no run in the study lacks its own evidence.

The NMS intervention likewise has no proposal-stage option. The runner assigns only `roi_heads.nms_thresh`, leaving the region proposal network untouched, and each Faster R-CNN manifest records `"rpn_nms_changed": false`. The isolation of final RoI suppression from proposal generation is therefore verifiable from the saved metadata rather than resting on the description in this report.

2.5 Software environment

Component	Version	Component	Version
Python	3.12.3	pycocotools	2.0.11
PyTorch	2.8.0+cu128	tidecv	1.0.1
CUDA runtime	12.8	ultralytics	8.3.0
Baseline GPU	RTX A6000 48 GB	transformers	4.45.2

Table 3. Software and hardware baseline matrix for the eight baseline runs.

3. Baseline Cross-Domain Evaluation

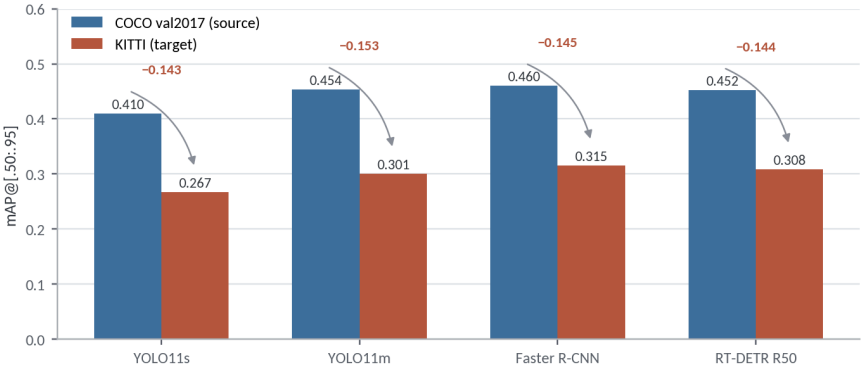


Figure 1. Source-domain and target-domain mAP@[.50:.95] for all four configurations. The annotated value above each pair is the per-model transfer gap, computed from unrounded run outputs.

Figure 1 shows a transfer gap that is remarkably consistent across architectures. Target-domain mAP decreases by 0.143 for YOLO11s, 0.153 for YOLO11m, 0.145 for Faster R-CNN, and 0.144 for RT-DETR. That is a spread of under one mAP point across three detector families with different backbones, necks, heads, assignment strategies, and post-processing regimes. Faster R-CNN attains the highest baseline KITTI mAP at 0.315, narrowly ahead of RT-DETR at 0.308 and YOLO11m at 0.301. The margin between the top three is smaller than the within-family gap between YOLO11s and YOLO11m, which is itself a result: model capacity inside a family moves the target metric more than the choice of family does. These values do not establish any architecture as universally better, since they are bounded to this three-class, zero-shot, 1,000-image protocol.

Model	COCO mAP	KITTI mAP	AP50	AP75	AP small / med / large	AP person / car / truck
YOLO11s	0.410	0.267	0.483	0.271	0.176 / 0.257 / 0.380	0.247 / 0.446 / 0.107
YOLO11m	0.454	0.301	0.536	0.312	0.180 / 0.292 / 0.423	0.256 / 0.463 / 0.186
Faster R-CNN	0.460	0.315	0.555	0.334	0.161 / 0.311 / 0.420	0.260 / 0.473 / 0.211
RT-DETR R50	0.452	0.308	0.531	0.315	0.217 / 0.303 / 0.426	0.258 / 0.492 / 0.175

Table 4. Baseline accuracy metrics. All target-domain values are on the KITTI evaluation subset.

Across all four target-domain models, cars are substantially easier than persons and trucks. The lowest target values occur for YOLO11s on trucks (0.107) and on small objects (0.176). The scale ordering is consistent, with large objects easier than medium and medium easier than small, which is the expected pattern for detectors transferred to a domain whose objects are on average farther from the camera. RT-DETR is the notable exception in degree rather than direction: its small-object AP of 0.217 is the strongest in the comparison despite its receiving one of the smaller input tensors, a point developed in Section 4.

3.1 Corroboration against the course reference harness

The course ngdet tutorial reports the same broad pattern on a separate 120-image driving3 evaluation: Faster R-CNN 0.279, RT-DETR 0.262, and YOLO11x 0.260 on KITTI [10]. The two protocols differ in taxonomy, model sizes, image count, score floor, and GPU, so these are not a numerical reproduction and are not presented as one. What does agree is the structure of the result: the same relative ordering of families, and the same compression of the field in the target domain. The present study observes COCO-to-KITTI drops spanning only 0.143 to 0.153 across three detector families, which independently corroborates the reference observation that domain shift dominates architecture choice under zero-shot transfer. This audit extends that observation by testing whether the residual gap can be attributed to final-stage duplicate suppression.

3.2 Source-domain qualitative check

Aggregate metrics can conceal a harness defect that still produces plausible numbers, so one qualitative panel is included as a traceability check on the saved predictions themselves. The image is selected by a declared deterministic rule rather than by inspection: the first saved record containing at least three shared-taxonomy ground-truth boxes, at least two shared classes, and at least two predictions at score 0.25 or above. The panel is drawn by a renderer that performs no inference and reads only the ground truth and the prediction boxes already stored in the YOLO11s COCO run record, so what is displayed is the same data that produced the metrics in Table 4.



Figure 2. Source-domain qualitative sanity check for saved YOLO11s predictions on COCO val2017 image 000000001532. The image was selected by a declared deterministic rule: the first saved record with at least three shared-taxonomy ground-truth boxes, two shared classes, and two predictions at score >= 0.25. Left: ground truth. Right: saved YOLO11s predictions. This figure illustrates the actual detection output but was not used to select quantitative results or to make a target-domain claim.

The 0.25 score threshold is used only to make the displayed boxes legible; quantitative evaluation retained predictions from the 0.001 score floor, so the panel shows a confident subset of what the metrics scored. Two source-domain observations follow, neither bearing on the cross-domain result. The shared-taxonomy classes are recovered with the box geometry the ground truth specifies, which is the behavior the harness must exhibit before any transfer claim is meaningful. The single truck annotation is recovered at 0.54 while an adjacent vehicle is scored as a car, a concrete instance of the car-truck boundary described in Section 2.2 and qualified in Section 7.

4. Architecture and Efficiency Comparison

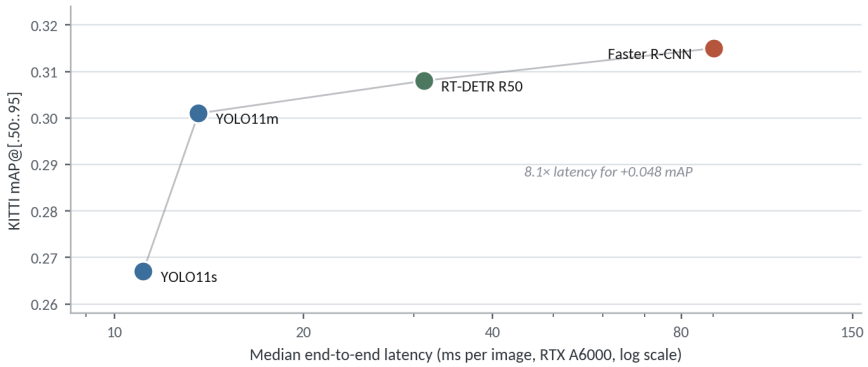


Figure 3. Accuracy against median end-to-end latency on the baseline RTX A6000 runs. Latency axis is logarithmic; only baseline runs appear, so no cross-GPU comparison is implied.

The baseline timing study shows the expected practical trade-off, spanning roughly an order of magnitude in latency for a 0.048 mAP range in accuracy. YOLO11s is fastest at 11.1 ms per image, about 90 frames per second, and uses only 59 MiB of peak allocated CUDA memory, but records the lowest target mAP. Faster R-CNN is the most accurate configuration in this protocol at the cost of 90.1 ms median latency and 439 MiB of peak allocated memory, an eight-fold latency penalty and a seven-fold memory penalty relative to YOLO11s for an mAP gain of 0.048. RT-DETR occupies the middle of the curve at 31.1 ms and 298 MiB while requiring no final NMS at inference. All timings include model preprocessing and post-processing within the same single-image evaluation harness and are not vendor benchmark claims.

Model	KITTI mAP	AR@100	Median ms	FPS	Peak MiB
YOLO11s	0.267	0.462	11.1	87.4	59
YOLO11m	0.301	0.501	13.6	71.5	115
Faster R-CNN	0.315	0.581	90.1	10.9	439

Model	KITTI mAP	AR@100	Median ms	FPS	Peak MiB
RT-DETR R50	0.308	0.584	31.1	31.4	298

Table 5. Accuracy, recall ceiling, latency, throughput, and peak memory on the baseline A6000 runs.

The recall column of Table 5 separates the two accurate configurations from the two fast ones more sharply than mAP does. Faster R-CNN and RT-DETR reach AR@100 values of 0.581 and 0.584 against 0.462 for YOLO11s, meaning the gap is substantially a recall ceiling rather than a ranking deficiency. A detector that never proposes an object cannot be recovered by any post-processing change, which is directly relevant to the intervention in Section 6.

The comparison also clarifies what is and is not held constant. Architecture-specific preprocessing is preserved by design, so the efficiency analysis is a practical system comparison rather than a claim that all architectures receive identical internal feature resolution. Input resolution alone does not, however, explain the scale-conditioned ordering. Faster R-CNN receives the largest native input yet records the lowest target AP-small at 0.161, while RT-DETR at 640 × 640 records the highest at 0.217. If pixels on target dominated small-object performance, that ordering would be inverted. The resolution difference remains a documented limitation, but it is not sufficient to account for the observed small-object result.

5. Failure Anatomy

TIDE decomposes the gap between measured and perfect average precision into mutually exclusive error types, each expressed as the mAP that would be recovered by fixing that error class alone. On KITTI, background and localization errors dominate the visible error anatomy for every configuration, while duplicate error is comparatively small at the baseline thresholds. This is the observation that motivates the intervention in Section 6 rather than assuming that NMS is the root cause. It is worth stating what the duplicate term can and cannot support: TIDE duplicate dAP is observational, measuring errors that survived post-processing, and is not a causal estimate of how NMS behaves. A near-zero duplicate term at the default threshold is evidence that little duplicate headroom exists, which is itself a prediction the sweep can test directly.

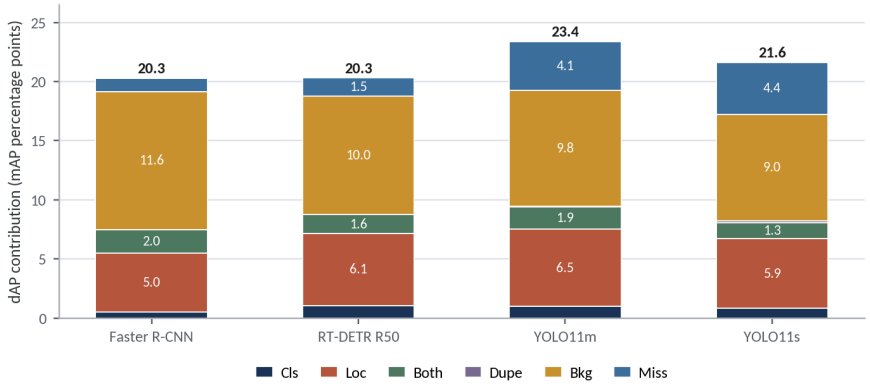


Figure 4. Target-domain TIDE error decomposition for the four baseline runs. Segment labels are omitted below 1.2 points for legibility. The dAP axis is measured in mAP percentage points.

Model	Cls	Loc	Both	Dupe	Bkg	Miss	Total
Faster R-CNN	0.52	4.97	1.96	0.03	11.64	1.16	20.29
RT-DETR R50	1.06	6.09	1.58	0.04	9.97	1.55	20.29
YOLO11m	0.99	6.53	1.87	0.07	9.82	4.08	23.35
YOLO11s	0.81	5.92	1.33	0.15	9.02	4.37	21.61

Table 6. TIDE error components in mAP percentage points on the KITTI evaluation subset.

Table 6 makes the decisive point quantitatively. Duplicate error accounts for between 0.03 and 0.15 mAP points out of total error budgets of 20.3 to 23.4 points, which is 0.16 percent of measured error for Faster R-CNN and 0.70 percent for YOLO11s. Background error is between 9.0 and 11.6 points, and localization error between 5.0 and 6.5 points, so each is one to two orders of magnitude larger than the duplicate term. The headroom available to any duplicate-suppression change was therefore already near zero before the sweep began, which is a stronger and more direct result than the sweep itself can provide: a threshold cannot recover error that does not exist.

The Miss component separates the field in a way the aggregate metric conceals. Faster R-CNN and RT-DETR record 1.16 and 1.55 points of missed-detection error, while YOLO11m and YOLO11s record 4.08 and 4.37. That ordering

reproduces the AR@100 ordering in Table 5 exactly, where the same two configurations reach 0.581 and 0.584 against 0.501 and 0.462. Two independent measurements therefore agree that a substantial share of the YOLO deficit in this protocol is a recall ceiling rather than a ranking or filtering problem. Objects a detector never proposes cannot be returned by any post-processing setting, which further constrains what the Section 6 intervention could possibly achieve.

Two secondary observations are worth recording. Faster R-CNN carries the largest background component at 11.64 points despite attaining the highest target mAP, which is consistent with a high-recall detector producing more low-score candidates that fall outside the restricted ground truth; Section 7.1 explains why this component should be read as an upper bound. RT-DETR also registers a small but non-zero duplicate term of 0.04 points despite applying no final NMS, confirming that one-to-one Hungarian matching suppresses duplicates structurally but not perfectly at inference time.

Four ground-truth stratifications support the analysis. The scale bands follow COCO area conventions computed in original image coordinates. The two density stratifications are deliberately distinct: the same-class bands measure the maximum intersection over union between an object and its nearest neighbor of the same class, while the all-class bands use the nearest spatial neighbor of any class. That distinction matters because class-aware NMS acts only within a class, whereas cross-class proximity can still cause occlusion and feature ambiguity without being directly suppressed. Separating the two makes it possible to test whether a post-processing intervention acts through the channel it is theorized to act through, which Section 6 does.

Ground-truth stratum	Count	Definition
small / medium / large	790 / 2,416 / 1,281	COCO area bands in original image coordinates
same-class isolated / near / overlap	2,579 / 1,102 / 806	Max same-class IoU < .10 / [.10, .30) / ≥ .30
all-class isolated / near / overlap	2,472 / 1,187 / 828	Same bands using any-class spatial neighbor
person / car / truck	646 / 3,685 / 156	Strict shared taxonomy only

Table 7. Target-domain ground-truth strata and their definitions.

The class distribution is heavily imbalanced toward cars, at 3,685 instances against 646 persons and 156 trucks. Truck conclusions are therefore treated as evidence from the declared fixed subset rather than as a population estimate, and the aggregate mAP is dominated by car performance to a degree the per-class column in Table 4 makes visible.

6. NMS Intervention

6.1 Design

The intervention holds model weights, input images, score floor, and saved-candidate cap fixed while sweeping only the final detection NMS IoU threshold across five values from 0.30 to 0.90. Because every other factor is fixed and the pipeline is deterministic, the observed accuracy response is attributable to the swept threshold alone. For Faster R-CNN the region proposal network's suppression threshold is held at its default throughout, so only the final RoI suppression varies; this isolates the post-processing stage from proposal generation, which would otherwise change the candidate set rather than its filtering. RT-DETR is excluded from the sweep because it applies no final NMS. The analogous lever for a set-prediction model would be top-k query selection, which was deliberately not swept in order to keep the intervention single-factor.

6.2 Accuracy response

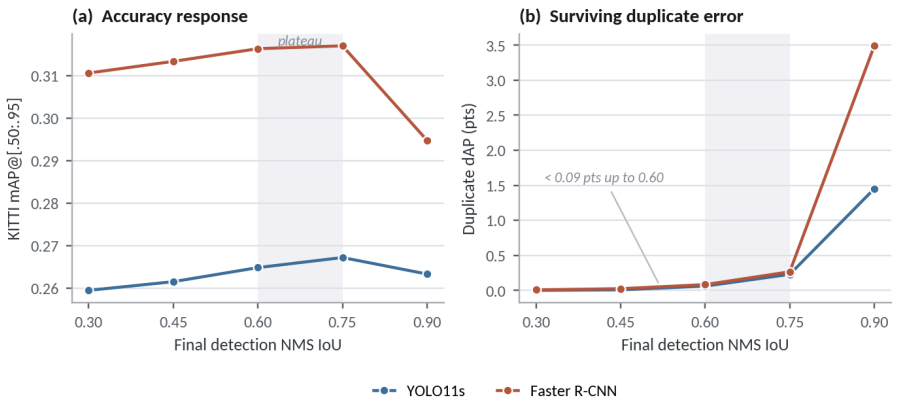


Figure 5. Left: KITTI mAP@[.50:.95] against final detection NMS IoU. Right: TIDE duplicate dAP for the same runs. The shaded band marks the flat intermediate plateau between 0.60 and 0.75.

Model	NMS 0.30	0.45	0.60	0.75	0.90	Baseline
YOLO11s	0.259	0.262	0.265	0.267	0.263	0.267 at 0.70
Faster R-CNN	0.311	0.313	0.316	0.317	0.295	0.315 at 0.50

Table 8. Target-domain mAP across the final-stage NMS IoU sweep, with baseline settings for reference.

The largest observed point estimates occur at 0.75, at 0.267 for YOLO11s and 0.317 for Faster R-CNN. These should not be read as a resolved optimum. The differences from the 0.60 setting are only 0.0024 and 0.0007 respectively, and this study does not estimate confidence intervals, so the defensible interpretation is a flat intermediate-threshold plateau spanning 0.60 to 0.75 rather than a located maximum. Both configurations decline at 0.90, where duplicate dAP rises sharply. The shape of the curve is therefore consistent throughout: suppression that is too aggressive removes correct detections along with duplicates, suppression that is too permissive admits duplicates that cost precision, and a wide region between the two produces almost identical aggregate accuracy.

6.3 Mechanism: density-stratified recall

Aggregate mAP alone cannot distinguish between a threshold change that acts through duplicate suppression and one that acts through some unrelated channel. The density stratification from Section 5 provides the test. Using score-ordered one-to-one matching at IoU 0.50 and the standard 100-detection cap, same-class recall was recomputed within each density band across all sweep runs.

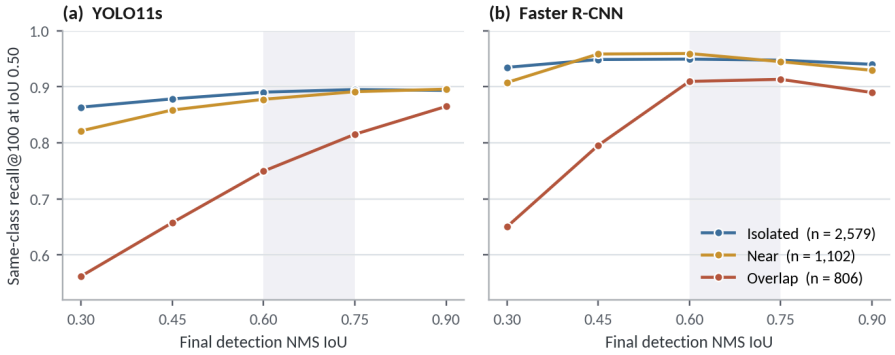


Figure 6. Same-class stratum recall@100 at IoU 0.50 against final detection NMS IoU. Stratum instance counts are constant across thresholds. Note the shared vertical scale.

Model	Stratum	0.30	0.45	0.60	0.75	0.90	Δ 0.30→0.75
YOLO11s	Isolated (n=2,579)	0.864	0.878	0.890	0.895	0.893	+0.031
YOLO11s	Near (n=1,102)	0.821	0.858	0.877	0.891	0.896	+0.070
YOLO11s	Overlap (n=806)	0.562	0.658	0.749	0.815	0.865	+0.253
Faster R-CNN	Isolated (n=2,579)	0.934	0.948	0.949	0.947	0.940	+0.013
Faster R-CNN	Near (n=1,102)	0.907	0.958	0.959	0.945	0.929	+0.037
Faster R-CNN	Overlap (n=806)	0.650	0.795	0.909	0.913	0.890	+0.263

Table 9. Same-class stratum recall@100 at IoU 0.50 across the full NMS sweep. Overlap recall responds eight to twenty times more strongly than isolated recall over the plateau.

The separation is unambiguous and monotonic. Relaxing suppression from 0.30 to 0.75 raises YOLO11s overlap recall from 0.562 to 0.815, a gain of 0.253, while isolated recall moves only from 0.864 to 0.895. Faster R-CNN shows the same signature, with overlap recall rising from 0.650 to 0.913 against an isolated change from 0.934 to 0.947. The near band behaves as an intermediate case in both models, which is the expected ordering if the effect is driven by same-class spatial proximity. The intervention therefore acts through precisely the channel it was theorized to act through: class-aware suppression removes correct detections of objects that overlap another instance of their own class, and loosening the threshold returns them.

The two panels of Figure 5 together explain why this recall gain does not translate into a proportional accuracy gain. Duplicate dAP stays below 0.09 points for both models at every threshold up to 0.60, rises to roughly 0.23 and 0.27 points at 0.75, and then jumps to 1.45 and 3.49 points at 0.90. Overlap recall continues to improve for YOLO11s at 0.90, reaching 0.865, yet aggregate mAP declines, because the duplicates admitted alongside those recovered detections cost more precision than the additional recall returns. Recall improvement alone is consequently not a valid criterion for choosing an operating threshold, and the 0.90 point is the clearest demonstration of that in the study.

The mechanism result and the bound result are complementary rather than contradictory. The threshold does what it is supposed to do, and it does it efficiently within the overlap stratum. That stratum simply contains 806 of 4,487 target instances, so even a large proportional recall gain inside it translates into a small movement in aggregate mAP. Identifying the mechanism is what makes the small aggregate effect interpretable instead of merely disappointing.

6.4 Per-class response and within-model cost

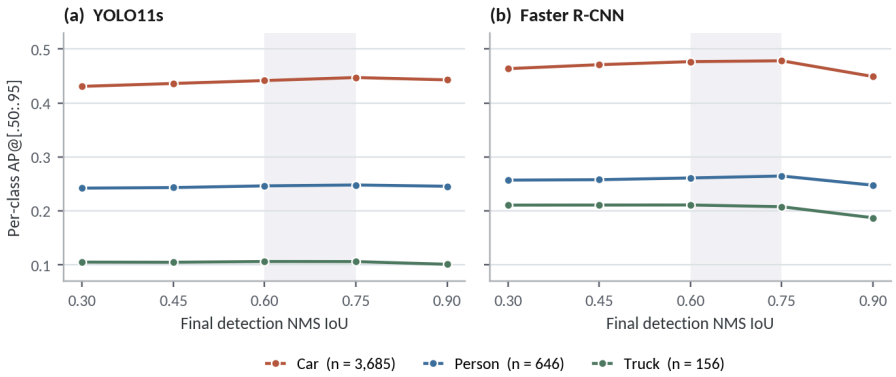


Figure 7. Per-class AP across the NMS sweep. Class instance counts are shown in the legend; the truck series rests on 156 instances and is interpreted with corresponding caution.

The per-class decomposition is consistent with the stratified mechanism. Car and person average precision improve through 0.75 for both configurations, with Faster R-CNN car AP rising from 0.464 to 0.478 and person AP from 0.257 to 0.265, before both fall at 0.90. Truck average precision is essentially flat through 0.60 and then declines, gaining nothing from the relaxation that benefits the other two classes. That pattern is what the overlap mechanism predicts: trucks are sparse in this subset at 156 instances and rarely appear adjacent to another truck, so the same-class overlap stratum that the threshold acts upon contains few of them. Because of the small count and the COCO-to-KITTI truck boundary described in Section 2.2, this is recorded as a consistent observation rather than a headline mechanism claim.

Peak allocated memory is unchanged across the sweep for both configurations, at approximately 59 to 71 MiB for YOLO11s and a constant 439 MiB for Faster R-CNN, which is expected because the threshold alters filtering rather than allocation. No monotonic latency trend is established from these single-pass measurements, and the sweep runs executed on the recovery GPU are not compared against the baseline timing study in Section 4. The defensible conclusion is narrow: within the measured range, the choice of final NMS threshold carries no observable memory cost and no demonstrated latency cost.

7. Limitations

7.1 Taxonomy and ignore-region handling

This is a zero-shot study and not a fine-tuning study, and it uses a strict shared taxonomy, so non-target classes are excluded from the three-class evaluator. Excluded KITTI labels are removed from ground truth rather than retained as ignore regions. A prediction overlapping a removed object can therefore be counted as a background false positive even when it correctly identifies a real object. This is a deliberate closed-set choice, because mapping a Van to either car or truck would itself be a heuristic with no defensible ground truth, but it has a measurable consequence: the TIDE background component should be read as an upper bound on detector hallucination rather than a clean estimate of it. Cyclist and Person_sitting are the strongest candidates for a future ignore-region sensitivity analysis, since a person on a bicycle and a seated person are both unambiguously persons under the COCO definition, unlike the genuinely ambiguous Van case.

COCO crowd annotations in the target classes were likewise dropped rather than retained as COCO ignore regions. Results are consequently strict three-class audit metrics and are not comparable to official full-category COCO average precision. A sensitivity re-audit with retained ignore regions would require the original annotation files, because discarded boxes are not present in the saved prediction records, and would need to be reported as separately labeled results rather than folded into the present numbers.

7.2 Protocol and measurement limitations

Native preprocessing differs by model family and is documented rather than forced to a single resize recipe, so the accuracy-latency comparison is checkpoint-native and not resolution-matched. Unlike the official KITTI benchmark, this audit scores every mapped object regardless of truncation, occlusion, or box height, so small-object results are intentionally not comparable to KITTI leaderboard difficulty tiers. The truck class has only 156 target instances alongside the definitional mismatch described in Section 2.2. The deterministic 1,000-image subsets support reproducibility and matched comparison but are not full-dataset leaderboard evaluations. No confidence intervals are

estimated, so differences on the order of 0.001 to 0.002 mAP are reported as a plateau rather than as an ordering. Finally, the NMS sweep and the baseline speed study were run on different GPU models, so their interpretations are kept separate throughout.

8. Conclusions

The cross-domain drop is not reducible to duplicate suppression, and two independent lines of evidence agree on that. First, the TIDE decomposition shows duplicate error accounting for 0.03 to 0.15 mAP points against total error budgets of 20.3 to 23.4 points, which is under one percent of measured error at baseline thresholds; the headroom for duplicate recovery did not exist before the intervention began. Second, the sweep confirms the bound directly: YOLO11s gains 0.77 mAP points and Faster R-CNN gains 0.64 points from NMS 0.30 to 0.75, against COCO-to-KITTI drops of 14.29 and 14.55 points respectively. This bounds the observed final-stage NMS effect to approximately five percent of the domain gap in this protocol, with background, localization, and missed detections accounting for the remainder.

The stratified analysis identifies where the recovered accuracy comes from. Relaxed suppression acts almost entirely on same-class overlapping objects, raising overlap recall by roughly 0.25 for both swept configurations while leaving isolated recall nearly unchanged. The post-processing lever is therefore correctly targeted but structurally limited: it addresses one failure channel that accounts for a minority of target instances, and it cannot touch the missed detections and localization errors that constitute the bulk of the gap. The practical implication for a practitioner transferring a COCO-pretrained detector to a driving domain is that threshold tuning is worth doing once, is insensitive across a wide plateau, and should not be expected to substitute for domain adaptation.

A useful next experiment would fine-tune one configuration on the target training split and repeat the same TIDE, stratification, and NMS protocol, with bootstrap confidence intervals over the image sample and an explicit ignore-region sensitivity analysis. That design would test whether the error composition shifts away from background and missed detections once the domain gap itself is addressed, which is the hypothesis the present bound most directly motivates.

9. Code, Configuration, and Evidence

All experiment code, configuration, command logs, and compact result tables are available at **github.com/harshitxjangam/cross-domain-detection-audit**. The repository contains the evaluation package under `src/detaudit/`, the split generator and cloud commands under `scripts/cloud/`, the analysis and figure build under `scripts/analysis/`, the versioned experiment definitions under `configs/`, the exact completed command patterns under `commands/`, taxonomy, reproducibility and runtime notes under `docs/`, and CPU-only checks under `tests/`.

Raw prediction records and dataset images are intentionally excluded from version control because of their size. Each of the eighteen runs retains a manifest recording its resolved arguments, the evaluated image identifier list, library versions, and a SHA-256 hash of its saved prediction records, and those hashes are reproduced in the compact summary table in the repository. Hash equality is expected only on identical hardware and software, since TF32 convolution behavior differs across GPU generations, so the manifests record the device for every run. Every figure and table in this report is generated by `scripts/analysis/build_submission.py` directly from the saved artifacts, so no value in this report is transcribed by hand.

References

[1] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. Advances in Neural Information Processing Systems (NeurIPS), 2015. arXiv:1506.01497.

[2] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko. End-to-End Object Detection with Transformers. European Conference on Computer Vision (ECCV), 2020. arXiv:2005.12872.

[3] W. Lv, Y. Zhao, S. Xu, J. Wei, G. Wang, C. Cui, Y. Du, Q. Dang, and Y. Liu. DETRs Beat YOLOs on Real-time Object Detection. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024. arXiv:2304.08069.

[4] D. Bolya, S. Foley, J. Hays, and J. Hoffman. TIDE: A General Toolbox for Identifying Object Detection Errors. European Conference on Computer Vision (ECCV), 2020. arXiv:2008.08115.

[5] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common Objects in Context. European Conference on Computer Vision (ECCV), 2014. arXiv:1405.0312.

[6] A. Geiger, P. Lenz, and R. Urtasun. Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2012.

[7] Ultralytics. YOLO11 model documentation. https://docs.ultralytics.com/models/yolo11/

[8] TorchVision Contributors. Models and pre-trained weights: object detection. Official documentation. https://docs.pytorch.org/vision/stable/models.html

[9] Hugging Face. RT-DETR, Transformers documentation. https://huggingface.co/docs/transformers/model_doc/rt_detr

[10] K. Liu. DeepDataMiningLearning, ngdet tutorial. https://github.com/lkk688/DeepDataMiningLearning